TURFLY

Complete Project Explanation and Viva Guide

Spring Boot + MongoDB + React + JWT + Brevo + SSLCommerz

A teacher-ready guide to the architecture, features, request flows, security, database, frontend, testing, deployment, and every authored code file.

Prepared from the current Turfly repository - October 2026

How to use this guide

This guide explains the code at two levels. First, it explains complete user journeys such as registration, booking, payment, and admin approval. Second, the file-by-file reference explains each authored file using exact line ranges. A line range is used when several consecutive lines form one Java or React statement; separating those lines would make the explanation harder, not easier.

Important: Real credentials are not printed. Configuration is explained with environment-variable names only.

- Before a viva, read Chapters 1-5 and the common questions at the end.
- When a teacher names a class, use the file-by-file chapter and follow its line ranges.
- When asked how a feature works, describe the request flow: React page -> API helper -> REST controller -> service -> repository -> MongoDB -> DTO response.
- Generated Vite bundles, package-lock.json, and PNG images are identified separately because they are outputs/assets, not handwritten business logic.

Contents

- 1. Project summary and goals
- 2. Architecture and request lifecycle
- 3. Database design
- 4. Feature workflows
- 5. Security and reliability
- 6. REST API map
- 7. Frontend behavior and UI design
- 8. Testing strategy
- 9. Deployment
- 10. File-by-file and line-range explanation
- 11. Viva questions and answers
- 12. Demonstration checklist and honest limitations

1. Project summary and goals

Turfly is a full-stack sports turf discovery and booking platform for football and cricket venues in Niketon, Dhaka. A public visitor can explore pitches, ratings, prices, venue information, availability, and the contact form. A registered user can reserve a 90-minute slot, apply a coupon, pay through SSLCommerz, track approval, print a receipt, review completed sessions, edit a profile, reset a password, or delete an account. An administrator manages bookings, pitches, images, coupons, contact messages, users, and summary statistics.

Why the architecture is deliberately simple

The project follows the familiar Controller-Service-Repository structure. Controllers only translate HTTP requests and responses. Services contain rules. Repositories contain MongoDB queries. DTOs define safe API shapes. Models represent stored documents. This separation is enough for a classroom project without adding complex domain layers, mappers, commands, events, or microservices.

Layer	Responsibility	Example
React UI	Collect input, show state, prevent duplicate clicks	BookingPage, Admin, Dashboard
API helper	Attach JWT, send fetch request, normalize errors	frontend/src/api.js
REST controller	Map URL and HTTP method, validate DTO	AuthController
Service	Enforce business rules and coordinate work	BookingService
Repository	Read and write MongoDB documents	BookingRepository
MongoDB	Persist users, pitches, bookings and supporting data	turfly database

Technology choices

Technology	Why it is used
Spring Boot 3.5	REST API, dependency injection, validation, configuration, scheduling and static file serving.
MongoDB	Document storage fits the simple entity shapes and Spring Data generates most queries.
Spring Security + JWT	Stateless authentication and role-based protection for a separate frontend.
React 18 + Vite	Component-based UI with a fast build and simple Vercel deployment.
Caffeine cache	Five-minute in-memory caching for public pitches and admin statistics.
Spring AI	Optional OpenAI assistant with a deterministic offline fallback.
Brevo HTTPS API	Registration OTP and reset email on a host where SMTP ports may be restricted.
SSLCommerz	Bangladesh-friendly hosted sandbox payment and server-side validation.

2. Architecture and request lifecycle

High-level architecture

Browser / React on Vercel
HTTPS fetch + JWT
Spring Boot REST API on Render
Controller -> Service -> Repository
MongoDB Atlas

Normal authenticated request

- The user logs in. AuthService verifies BCrypt credentials and JwtService creates a signed token.
- AuthContext stores the token in localStorage and keeps the safe user DTO in React state.
- api.js reads the token and adds Authorization: Bearer <token>.
- JwtAuthenticationFilter verifies the token, loads UserDetails, and fills the SecurityContext.
- SecurityConfig allows or denies the endpoint based on authentication and role.
- The controller validates the request DTO and calls a service.
- The service enforces ownership, state, timing, price, or payment rules and uses a repository.
- The controller returns a DTO. The frontend updates state and releases its loader in finally.

Local and deployed modes

Locally, Vite can proxy /api to port 9091 during development. A production Vite build can also copy into Spring Boot static resources, so IntelliJ serves the complete application from one port. In production, Vercel serves React and Render serves Spring Boot; VITE_API_URL points the browser to Render, FRONTEND_URL controls CORS and reset links, and BACKEND_URL controls payment callbacks and uploaded image URLs.

3. Database design

Collection	Important fields	Reason
users	email, name, phone, password, role, enabled, resetTokenHash, resetTokenExpiresAt	Identity, authorization and reset state.
registration_verifications	email, pending profile, passwordHash, otpHash, expiresAt, attempts	Prevents account creation before email ownership is proven.
pitches	name, sport, description, pricePerHour, capacity, imageUrl, features, active	Public inventory and admin-managed venue information.
bookings	customer snapshot, pitch snapshot, schedule, amounts, status, payment references	Preserves what was booked even if profile or pitch later changes.
coupons	code, type, value, expiry, maximum and used counts, active	Controlled promotions.

Collection	Important fields	Reason
reviews	unique bookingId, pitchId, userId, rating, comment	One verified review per completed booking.
contact_messages	sender details, subject, message, status, note	Admin support inbox.

Important data-design decisions

- User email and coupon code are indexed and unique.
- A booking stores user and pitch names as snapshots for receipts and history.
- Passwords use BCrypt. OTPs and reset tokens use SHA-256 hashes; plain values exist only long enough to email them.
- Booking and payment statuses are separate. PAID does not automatically mean CONFIRMED.
- Spring Data auditing fills createdAt and updatedAt.
- Repository method names become MongoDB queries without handwritten query syntax.

4. Feature workflows

4.1 Registration with email OTP

- AuthPage sends name, email, phone and password to POST /api/auth/register.
- Bean Validation checks lengths, email format, phone format and password length.
- AuthService normalizes the email and rejects an existing user.
- SecureRandom creates a six-digit code. BCrypt hashes the pending password and SHA-256 hashes email:OTP.
- Only a RegistrationVerification document is stored; no User exists yet.
- MailService selects Brevo when configured and sends HTML plus plain text.
- The UI changes to a one-time-code input with resend and change-details controls.
- Verification checks expiry and at most five wrong attempts. Correct OTP creates the enabled USER, removes pending data and returns JWT plus user DTO.

Teacher answer: The account is not created before verification. The pending password is already BCrypt-hashed, and the OTP is stored only as a SHA-256 hash.

4.2 Login and JWT

AuthenticationManager asks CustomUserDetailsService for the user and BCrypt compares the supplied password with the stored hash. JwtService signs a token containing email subject, user ID, role, issue time and expiry. The backend is stateless: it does not store an HTTP session. Each protected request brings the token again.

4.3 Forgot password

The endpoint returns the same message whether the email exists or not, preventing account discovery. For an existing user it generates two UUIDs, stores their SHA-256 hash with a 30-minute expiry, and emails the original token inside a FRONTEND_URL reset link. Reset hashes the presented token, validates expiry, BCrypt-hashes the new password, and clears the token so it cannot be reused.

4.4 Availability and booking

BookingService owns twelve legal start times. Every slot is 90 minutes. Availability rejects past times in Asia/Dhaka and checks whether a non-cancelled booking already uses the pitch/date/start triple. The create method repeats that check, computes hourly price x 1.5, applies an optional coupon, and saves a user booking as PENDING_PAYMENT. synchronized is a simple collision guard for one application instance.

4.5 Coupon calculation

A blank coupon produces zero discount. Otherwise the service finds the case-insensitive code, checks active state, expiry and usage limit, then calculates a percentage or fixed discount. Discounts equal to or greater than the total are rejected. Usage increments only after verified payment, and couponCounted prevents duplicate callback increments.

4.6 SSLCommerz payment and admin approval

- PaymentService checks ownership and requires PENDING_PAYMENT and UNPAID.
- It POSTs store, amount, customer and callback data to the sandbox gateway.
- The frontend opens GatewayPageURL in a popup or uses same-tab fallback.
- A success callback is not trusted by itself. processPayment requires VALID status and matching transaction/amount.
- The service calls SSLCommerz validation API and verifies its status, transaction ID and amount.
- Only then does BookingService set PAID and AWAITING_APPROVAL.
- The result page tells the user payment succeeded but admin approval is pending.
- Admin confirm/reject endpoints require PAID plus AWAITING_APPROVAL.

4.7 Admin booking difference

If the logged-in role is ADMIN, normal booking creates CONFIRMED immediately and the React page skips checkout. The manual booking form also creates a confirmed reservation and optionally marks it paid with an ADMIN reference. This is useful for phone or walk-in reservations.

4.8 Pitch CRUD and uploads

Pitch create/update/delete endpoints are under /api/admin. Upload is separate: the browser first sends FormData, ImageStorageService validates size and type, writes a UUID filename, and returns an absolute public URL. The pitch request then stores that URL. Cache eviction makes changed names and prices appear on public pages immediately.

4.9 Reviews, receipts, contact, AI and account deletion

- ReviewService requires ownership, CONFIRMED state, a completed end time, and no existing review for the booking.
- ReceiptModal prints only .receipt-print with A4 print CSS, preventing duplicate/blank receipt pages.
- Contact messages have NEW, READ and RESOLVED status plus private admin notes.
- AiAssistantService uses Spring AI when enabled and deterministic venue answers otherwise.
- Account deletion verifies the password twice across a password modal and final confirmation, then deletes the user and clears local authentication.

5. Security and reliability

Control	Implementation	Why it matters
Password storage	BCryptPasswordEncoder	Slow salted hashes resist database-password recovery.
OTP/reset storage	SHA-256 hash plus expiry	Database does not contain usable codes or links.
JWT	HMAC signature, subject and expiry	Detects tampering and supports stateless requests.
Authorization	hasRole ADMIN and ownership checks	A token alone cannot perform every action.
Validation	Jakarta annotations on DTO records	Rejects malformed data before service logic.
Payment trust	Gateway validation API and amount/ID matching	Browser callback cannot mark arbitrary bookings paid.
Duplicate clicks	busy state + disabled LoadingButton	Reduces repeated POST/PUT/DELETE requests.
Error safety	GlobalExceptionHandler	Clients receive consistent messages; unexpected details stay in logs.
CORS	Frontend-only normal API; isolated callback rule	Limits browser origins without breaking hosted payments.

State machines to remember

```
Normal user booking:
PENDING_PAYMENT + UNPAID
  -> validated payment
AWAITING_APPROVAL + PAID
  -> admin confirm              -> admin reject
CONFIRMED + PAID                CANCELLED + PAID

Unpaid expiry/cancel:
PENDING_PAYMENT + UNPAID -> CANCELLED + UNPAID
```

6. REST API map

Method	Path	Access	Purpose
POST	/api/auth/register	Public	Request registration OTP
POST	/api/auth/register/verify	Public	Verify OTP and create account
POST	/api/auth/login	Public	Login and receive JWT
POST	/api/auth/forgot-password	Public	Request reset email
POST	/api/auth/reset-password	Public	Use token to set new password
GET	/api/auth/me	User	Load current safe profile
GET	/api/pitches[?sport=]	Public	List active pitches
GET	/api/bookings/availability	Public	One-day slots
GET	/api/bookings/availability-calendar	Public	Multi-day slots
POST	/api/bookings	User	Create reservation
GET	/api/bookings/me	User	Own bookings

Method	Path	Access	Purpose
DELETE	/api/bookings/{id}	User/owner	Cancel unpaid booking
POST	/api/payments/checkout/{id}	User/owner	Create SSLCommerz checkout
GET/POST	/api/payments/sslcommerz/{outcome}	Gateway	Payment callbacks
GET	/api/coupons/validate	User	Preview coupon
GET/POST	/api/reviews/...	Mixed	Read/create reviews
PUT/DELETE	/api/users/me...	User	Profile/password/delete
POST	/api/contact	Public	Submit contact message
POST	/api/ai/ask	Public	Ask assistant
GET/POST/PUT/DELETE	/api/admin/**	ADMIN	Management operations

7. Frontend behavior and design

The frontend uses React state rather than a large state-management library. Each page owns its form, loading, error and modal state. AuthContext is the only global application state because authentication is needed across pages. The custom router keeps the dependency list small and demonstrates browser navigation directly.

Loader and confirmation pattern

```
async function action() {
  if (busy) return;
  setBusy(true);
  try { await api(...); }
  catch (error) { setMessage(error.message); }
  finally { setBusy(false); }
}

<LoadingButton loading={busy}>Save</LoadingButton>
```

The first guard blocks a second click before React finishes the current request. LoadingButton also disables the DOM button and displays aria-busy. Destructive or important state transitions first set an object such as pitchToRemove or bookingDecision; ConfirmDialog uses that object to display context and performs the API call only after confirmation.

Responsive and animated design

- CSS variables define the dark surfaces, text, lime accent, errors, borders and shadows.
- The hero combines an image, gradients, grid overlay, football-pitch drawing, rotating football/bat orbits and floating cards.
- Reveal uses IntersectionObserver so cards animate only when entering the viewport.
- Media queries restructure navigation, grids, booking layout, admin layout and dialogs.
- prefers-reduced-motion disables animation for users who request it.
- @media print isolates one receipt and @page requests one A4 portrait page.

8. Testing strategy

The project currently runs 25 backend tests. Repositories and external systems are mocked so tests are fast and deterministic. The tests focus on business rules that would be costly or embarrassing to get wrong rather than testing Lombok getters or trivial controller delegation.

Test class	What it proves
BookingServiceTest	90-minute calculation, price, collisions, calendar, coupon, admin bypass, payment approval.
AuthServiceTest	No user before OTP; correct OTP creates user; wrong OTP does not.
MailServiceTest	OTP content/expiry and deployed reset URL.
SecurityConfigTest	Gateway callback origin is accepted while admin API stays frontend-only.
JwtServiceTest	Token subject and user validation.
CouponServiceTest	Percentage math, expired rejection and normalization.
ReviewServiceTest	Completed-session rule.
UserServiceTest	Wrong delete password rejects; correct one deletes.
ContactServiceTest	Status, resolved flag and trimmed note.
AiAssistantServiceTest	Useful answer without an external AI key.

```
mvn test
# Expected: Tests run: 25, Failures: 0, Errors: 0

cd frontend
npm run build
```

9. Deployment

Backend - Render

Render builds the repository Dockerfile. The first stage compiles the Spring Boot JAR with Maven and the second stage runs it on Java 21. Render supplies PORT and production environment variables. MongoDB Atlas is external. The backend production URL is used for REST requests, SSLCommerz callbacks, and uploaded image URLs.

Frontend - Vercel

Vercel builds the frontend folder with Vite. When VERCEL exists, vite.config.js writes to dist rather than Spring static resources. vercel.json rewrites all paths to index.html so browser refresh works on client routes. The production alias is https://turflyy.vercel.app.

Production variables by responsibility

Variable	Used for
MONGODB_URI	Atlas connection
JWT_SECRET	JWT signature
FRONTEND_URL	CORS, reset links and payment-result redirects
BACKEND_URL	SSLCommerz callbacks and upload URLs
VITE_API_URL	Browser API base
BREVO_API_KEY / BREVO_SENDER_EMAIL	Transactional email
SSLCOMMERZ_STORE_ID / PASSWORD	Sandbox or live checkout
OPENAI_API_KEY / AI_ENABLED	Optional AI responses

10. File-by-file and line-range explanation

Line numbers match the current repository at commit 0be17d2. Import-only lines are described together because their purpose is to make the types below available. Closing braces and blank lines only delimit the blocks described around them.

10.1 Root build and runtime files

pom.xml

Purpose: Maven project definition and dependency versions.

Lines 1-5: Declare Maven XML format and schema.

Lines 6-12: Use Spring Boot 3.5.16 parent defaults.

Lines 14-18: Set project coordinates, name and description.

Lines 20-25: Choose Java 21 and library versions.

Lines 27-38: Import the Spring AI BOM so AI modules share compatible versions.

Lines 40-102: Add web, MongoDB, security, validation, mail, cache, Caffeine, Spring AI, JWT, Lombok and test dependencies. JWT implementation/Jackson are runtime-only; tests are test-only.

Lines 104-139: Configure Lombok annotation processing, executable Spring Boot packaging, and Mockito Java-agent support for tests.

Dockerfile

Purpose: Two-stage production container for Render.

Lines 1-end: Build the JAR in a Maven/Java 21 image, copy only the result into a smaller Java 21 runtime image, expose the service port and run the JAR.

src/main/resources/application.properties

Purpose: Readable configuration with safe environment overrides.

Lines 1-6: Application name, optional .env import, Mongo URI/database and port/address.

Lines 8-11: Map simple Mongo names to standard Spring Data properties and enable indexes.

Lines 13-15: JWT secret and eight-hour default expiry.

Lines 17-26: Frontend/contact/admin values.

Lines 28-41: SMTP and Brevo provider configuration; secrets come from environment variables.

Lines 43-48: SSLCommerz store, sandbox/demo flags and public backend URL.

Lines 50-59: Disable unused AI model types and make chat optional.

Lines 61-70: Caffeine policy, development static-cache setting, multipart limit and upload folder.

Lines 71: Prevent default server exceptions from leaking messages.

README.md

Purpose: Developer overview, setup, endpoints and verification commands.

Lines 1-end: Documents requirements, configuration placeholders, local run, payment callbacks, endpoint groups and build checks. It is documentation, not runtime code.

10.2 Bootstrap and configuration classes

TurflyApplication.java

Purpose: Spring Boot entry point.

Lines 1-6: Package and framework imports.

Lines 8-10: Enable caching, scheduling and component auto-configuration.

Lines 11-14: main starts Spring and embedded Tomcat.

config/MongoConfig.java

Purpose: Enable auditing.

Lines 1-8: Configuration plus EnableMongoAuditing makes CreatedDate and LastModifiedDate work.

config/DataSeeder.java

Purpose: Create initial admin and pitches once.

Lines 1-19: Dependencies, Lombok and logging.

Lines 20-27: Repositories, encoder and configurable admin values.

Lines 29-37: Run after startup; database failure is logged without stopping the app.

Lines 39-49: Create BCrypt admin only when email does not exist.

Lines 51-60: Create three pitches only when collection is empty.

Lines 63-68: Shared builder fills location, price, capacity, image, features and active state.

config/SecurityConfig.java

Purpose: Stateless security, authorization, BCrypt and CORS.

Lines 1-24: Imports and security configuration annotations.

Lines 25-47: Disable CSRF for token API, enable CORS, make sessions stateless, list public routes, protect admin routes, and insert JWT filter.

Lines 50-58: Expose BCrypt encoder and AuthenticationManager beans.

Lines 60-79: Use an isolated open-origin callback CORS rule and a credentialed frontend-only rule for the rest.

Lines 80-81: Close class.

config/UploadWebConfig.java

Purpose: Serve uploaded files.

Lines 1-16: Read the configured upload directory.

Lines 18-23: Normalize its absolute URI and map /uploads/** to it.

10.3 Security classes

security/CustomUserDetailsService.java

Purpose: Adapt MongoDB User to Spring Security UserDetails.

Lines 1-13: Dependencies and injected repository.

Lines 14-23: Find normalized email, hide existence with a generic error, expose stored hash, enabled flag and ROLE_ authority.

security/JwtService.java

Purpose: Create and validate signed tokens.

Lines 1-24: Load HMAC key bytes and token lifetime.

Lines 26-36: Build token with email subject, ID/role claims, issue/expiry and signature.

Lines 38-40: Read subject.

Lines 42-50: Validate matching username and future expiry while safely rejecting JWT errors.

Lines 52-54: Central verified claims parser.

security/JwtAuthenticationFilter.java

Purpose: Authenticate every bearer-token request once.

Lines 1-20: Filter dependencies.

Lines 21-28: Skip when no Bearer header.

Lines 30-41: Extract email, load user, validate token, build authentication and store it in SecurityContext.

Lines 42-46: Ignore invalid token here and continue; protected routes are later denied by Security.

10.4 Models and enums

model/User.java

Purpose: User identity document.

Lines 1-end: Lines 11-18 map users and unique email; 20-26 store profile, BCrypt password, role/enabled and reset hash/expiry; 28-32 audit creation/update.

model/RegistrationVerification.java

Purpose: Pending OTP document keyed by email.

Lines 1-end: Lines 9-19 store pending profile, already-hashed password, OTP hash, expiry and failed attempts.

model/Pitch.java

Purpose: Bookable venue document.

Lines 1-end: Lines 13-25 define identity and public details; 27-31 safely default features and active state; 33-37 audit dates.

model/Booking.java

Purpose: Reservation and payment snapshot.

Lines 1-end: Lines 12-14 define collection/index/Lombok; 16-37 hold owner/customer/pitch/time/price/status/payment fields; 39-43 audit dates.

model/Coupon.java

Purpose: Promotion document.

Lines 1-end: Lines 12-24 define unique code and limits; 26-30 default use count and active state; 32-36 audit dates.

model/Review.java

Purpose: Verified pitch review.

Lines 1-end: Lines 10-17 define collection and unique booking; 19-26 store pitch/user/rating/comment/time.

model/ContactMessage.java

Purpose: Support inbox item.

Lines 1-end: Lines 10-20 store sender/message; 22-28 default workflow state and admin note; 30-31 creation time.

enums/BookingStatus.java

Purpose: Restrict state to known constants.

Lines 1-3: Package declaration and enum constants: PENDING_PAYMENT, AWAITING_APPROVAL, CONFIRMED, CANCELLED.

enums/PaymentStatus.java

Purpose: Restrict state to known constants.

Lines 1-3: Package declaration and enum constants: UNPAID, PAID, REFUNDED.

enums/Role.java

Purpose: Restrict state to known constants.

Lines 1-3: Package declaration and enum constants: USER, ADMIN.

enums/Sport.java

Purpose: Restrict state to known constants.

Lines 1-3: Package declaration and enum constants: FOOTBALL, CRICKET, MULTI_SPORT.

enums/DiscountType.java

Purpose: Restrict state to known constants.

Lines 1-3: Package declaration and enum constants: PERCENTAGE, FIXED.

enums/MessageStatus.java

Purpose: Restrict state to known constants.

Lines 1-3: Package declaration and enum constants: NEW, READ, RESOLVED.

10.5 DTO files

dto/AuthDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-7: Container class; private constructor prevents instances.

Lines 9-19: Validate registration and six-digit OTP requests.

Lines 21-31: Login, forgot and reset requests.

Lines 33-37: Safe user, authentication and message responses.

dto/BookingDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-11: Imports/container.

Lines 13-29: Validate user and manual booking inputs.

Lines 31-38: Complete booking response shape.

Lines 40-46: Slot, calendar and checkout response records.

dto/PitchDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-10: Imports/container.

Lines 12-22: Validate pitch create/update fields.

Lines 24-30: Public pitch and upload responses.

dto/CouponDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-10: Imports/container.

Lines 12-19: Validate coupon definition.

Lines 21-29: Admin coupon and user validation responses.

dto/ContactDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-9: Imports/container.

Lines 11-17: Validate public message.

Lines 19-27: Message response and admin update.

Lines 29: Public contact information.

dto/ReviewDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-10: Imports/container.

Lines 12-16: Validate booking ID, 1-5 rating and comment.

Lines 18-25: Individual and summary responses.

dto/UserDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-6: Container.

Lines 8-16: Profile and password-change validation.

Lines 18-20: Password payload for verification/deletion.

dto/AiDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-7: Container/imports.

Lines 9-10: Question limit and answer/AI-source response.

dto/AdminDtos.java

Purpose: Request/response records keep controllers safe and explicit.

Lines 1-8: Statistics response with users, pitches, bookings, confirmed and unread counts.

10.6 Repository files

repository/UserRepository.java

Purpose: Email/reset lookups, existence and enabled-user count.

Lines 1-end: Extends MongoRepository. Spring Data derives MongoDB queries from the method names, so no implementation class is needed.

repository/PitchRepository.java

Purpose: Active pitch queries ordered by name, optionally by sport.

Lines 1-end: Extends MongoRepository. Spring Data derives MongoDB queries from the method names, so no implementation class is needed.

repository/BookingRepository.java

Purpose: User/admin lists, slot collision query, transaction lookup and status count.

Lines 1-end: Extends MongoRepository. Spring Data derives MongoDB queries from the method names, so no implementation class is needed.

repository/CouponRepository.java

Purpose: Case-insensitive code lookup/existence and newest-first list.

Lines 1-end: Extends MongoRepository. Spring Data derives MongoDB queries from the method names, so no implementation class is needed.

repository/ReviewRepository.java

Purpose: One-review check and pitch/user lists.

Lines 1-end: Extends MongoRepository. Spring Data derives MongoDB queries from the method names, so no implementation class is needed.

repository/ContactMessageRepository.java

Purpose: Newest-first messages and unresolved count.

Lines 1-end: Extends MongoRepository. Spring Data derives MongoDB queries from the method names, so no implementation class is needed.

repository/RegistrationVerificationRepository.java

Purpose: Basic CRUD for pending email verification keyed by email.

Lines 1-end: Extends MongoRepository. Spring Data derives MongoDB queries from the method names, so no implementation class is needed.

10.7 Controller files

controller/AuthController.java

Purpose: REST boundary for this feature area.

Lines 11-16: Base /api/auth and injected services.

Lines 18-27: Request OTP then verify/create.

Lines 29-42: Login, forgot and reset.

Lines 44-47: Return current safe user.

controller/BookingController.java

Purpose: REST boundary for this feature area.

Lines 14-19: Base route and dependencies.

Lines 21-31: Public daily/calendar availability.

Lines 33-37: Create authenticated booking.

Lines 39-48: List/cancel own bookings.

controller/PaymentController.java

Purpose: REST boundary for this feature area.

Lines 13-18: Base route and dependencies.

Lines 20-29: Authenticated checkout and safe demo completion.

Lines 31-35: IPN form callback.

Lines 37-43: Success/fail/cancel validation and redirect.

controller/PitchController.java

Purpose: REST boundary for this feature area.

Lines 11-25: Public list/filter/get endpoints delegated to PitchService.

controller/AdminController.java

Purpose: REST boundary for this feature area.

Lines 21-32: Admin base route and injected feature services.

Lines 34-58: Stats, booking list/manual booking and decisions.

Lines 60-96: Messages, coupons and users.

Lines 98-118: Pitch CRUD and multipart image upload.

controller/UserController.java

Purpose: REST boundary for this feature area.

Lines 12-17: User base route/dependencies.

Lines 19-28: Profile and password change.

Lines 30-40: Delete-password verification and final deletion.

controller/ReviewController.java

Purpose: REST boundary for this feature area.

Lines 13-18: Base route/dependencies.

Lines 20-34: Public pitch summary, own reviews and authenticated creation.

controller/ContactController.java

Purpose: REST boundary for this feature area.

Lines 10-20: Public validated contact creation returning HTTP 201.

controller/CouponController.java

Purpose: REST boundary for this feature area.

Lines 8-17: Authenticated coupon preview for a pitch.

controller/AiController.java

Purpose: REST boundary for this feature area.

Lines 9-18: Public validated assistant question endpoint.

controller/PublicController.java

Purpose: REST boundary for this feature area.

Lines 7-16: Build immutable contact-info DTO from properties.

Lines 18-21: Return it publicly.

controller/HealthController.java

Purpose: REST boundary for this feature area.

Lines 8-13: Simple status/service/timestamp response for deployment checks.

controller/RootController.java

Purpose: REST boundary for this feature area.

Lines 6-15: Forward known SPA routes to index.html when Spring serves the frontend.

10.8 Service files

service/AuthService.java

Purpose: Business logic for this feature area.

Lines 23-35: Constants and injected dependencies.

Lines 37-60: Normalize registration, generate/hash/store OTP and send email with rollback on mail failure.

Lines 62-96: Verify pending request, expiry and attempts; create user only on success.

Lines 98-109: Authenticate login and issue JWT.

Lines 111-120: Create hashed 30-minute reset token without revealing account existence.

Lines 122-133: Validate reset token, change BCrypt password and make link single-use.

Lines 135-147: Email normalization and reusable SHA-256 helper.

service/MailService.java

Purpose: Business logic for this feature area.

Lines 14-26: Injected sender plus SMTP/Brevo/front-end settings.

Lines 28-49: OTP development fallback, content and provider routing.

Lines 51-65: Build environment-aware reset link and route provider.

Lines 67-79: Provider selection and SMTP sender.

Lines 81-97: Brevo reset HTML/text.

Lines 99-127: Validate Brevo settings and POST transactional email with safe error handling.

service/BookingService.java

Purpose: Business logic for this feature area.

Lines 17-29: Dhaka zone, twelve legal times and dependencies.

Lines 31-44: Daily and bounded calendar availability.

Lines 46-79: Collision-safe normal booking, price/coupon and admin shortcut.

Lines 81-99: Confirmed manual admin booking.

Lines 101-123: Lists, entity lookup and unpaid cancellation rules.

Lines 125-144: Idempotent generic/SSLCommerz paid transitions and coupon count.

Lines 146-166: Admin confirmation/rejection guards.

Lines 168-177: Expire abandoned payments.

Lines 179-191: Past/collision check and one-time coupon usage.

service/PaymentService.java

Purpose: Business logic for this feature area.

Lines 21-35: Dependencies and payment/public URL settings.

Lines 36-49: Ownership/state checks and optional demo URL.

Lines 51-87: Create gateway form, read response, save transaction and expose clear failure.

Lines 89-98: Safe demo completion.

Lines 100-110: IPN/callback handling and frontend redirect.

Lines 112-129: Validate status, IDs and amounts locally and server-to-server.

Lines 131-150: Validation API, sandbox host, booking lookup and defensive decimal parsing.

service/PitchService.java

Purpose: Business logic for this feature area.

Lines 14-30: Cached active list and individual lookup.

Lines 32-46: Create and evict caches.

Lines 48-61: Update all editable fields and evict caches.

Lines 63-70: Delete/lookup with not-found error.

service/CouponService.java

Purpose: Business logic for this feature area.

Lines 15-27: Preview final price for a pitch.

Lines 29-43: Validate coupon and compute discount.

Lines 45-51: Synchronized usage increment.

Lines 53-82: Admin list/create/update/delete.

Lines 84-100: Lookup, percentage guard, response, normalization and internal Discount record.

service/ReviewService.java

Purpose: Business logic for this feature area.

Lines 16-33: Ownership, status/time, duplicate checks and save.

Lines 35-45: Own list and public average summary.

Lines 47-56: Overnight-aware completion calculation and mapping.

service/ContactService.java

Purpose: Business logic for this feature area.

Lines 13-27: Normalize and save public message.

Lines 29-39: Admin list and quick resolve.

Lines 41-48: Update workflow status and trimmed note.

Lines 50-55: Count unresolved NEW messages.

service/UserService.java

Purpose: Business logic for this feature area.

Lines 14-24: Update name and phone.

Lines 26-31: Verify current password and save new BCrypt hash.

Lines 33-41: Password verification and deletion.

Lines 43-45: Admin safe-user list.

service/AiAssistantService.java

Purpose: Business logic for this feature area.

Lines 9-18: Optional ChatClient provider and enable flag.

Lines 20-38: Call AI with venue safety prompt or fall back after failure.

Lines 40-51: Deterministic keyword answers.

service/ImageStorageService.java

Purpose: Business logic for this feature area.

Lines 13-28: Allowed types, upload path and public backend base.

Lines 30-42: Reject empty/large/unsupported files using MIME then extension fallback.

Lines 43-51: UUID filename, create folder, copy bytes and return public URL.

service/AdminService.java

Purpose: Business logic for this feature area.

Lines 10-18: Repositories and cached stats method.

Lines 19-27: Count users, pitches, bookings, confirmed and new messages.

service/CurrentUser.java

Purpose: Business logic for this feature area.

Lines 10-19: Resolve Spring Authentication email back to the MongoDB User entity.

service/DtoMapper.java

Purpose: Business logic for this feature area.

Lines 10-15: Safe User -> UserResponse.

Lines 17-21: Pitch -> PitchResponse.

Lines 23-31: Booking -> response with compatibility defaults.

Lines 33-38: Contact -> response with legacy status fallback.

service/BookingCleanupJob.java

Purpose: Business logic for this feature area.

Lines 7-19: Every five minutes call expiry cleanup; tolerate unavailable MongoDB in frontend-only development.

10.9 Exceptions

exception/BadRequestException.java

Purpose: 400 invalid state/input.

Lines 1-5: Small RuntimeException subclass carrying a readable message.

exception/ConflictException.java

Purpose: 409 collision/duplicate.

Lines 1-5: Small RuntimeException subclass carrying a readable message.

exception/ForbiddenException.java

Purpose: 403 ownership/permission.

Lines 1-5: Small RuntimeException subclass carrying a readable message.

exception/NotFoundException.java

Purpose: 404 missing entity.

Lines 1-5: Small RuntimeException subclass carrying a readable message.

exception/GlobalExceptionHandler.java

Purpose: Convert failures to consistent JSON.

Lines 13-16: Advice and ApiError shape.

Lines 18-41: Map domain, access and upload exceptions to correct statuses.

Lines 43-49: Combine validation field errors.

Lines 51-55: Log unknown failures and hide internal details.

Lines 57-60: Build timestamped response.

10.10 Frontend foundation and components

frontend/src/main.jsx

Purpose: React frontend foundation/component.

Lines 1-6: Import React, root components and CSS.

Lines 8-14: Render App inside RouterProvider and AuthProvider under StrictMode.

frontend/src/App.jsx

Purpose: React frontend foundation/component.

Lines 1-14: Import layout and pages.

Lines 16-18: Read current route and decide standalone layout.

Lines 19-32: Map paths to pages/protection and provide 404.

Lines 33-38: Render navbar/footer/assistant only for normal pages.

frontend/src/router.jsx

Purpose: React frontend foundation/component.

Lines 1-13: Context, current location and popstate listener.

Lines 15-24: History navigation, scroll reset and provider.

Lines 26-28: Navigation hooks.

Lines 30-44: Accessible Link interception and active NavLink.

Lines 46-50: Effect-based redirect component.

frontend/src/api.js

Purpose: React frontend foundation/component.

Lines 1-7: Choose API base, read JWT, prepare headers and preserve FormData.

Lines 9-14: Fetch with friendly network error.

Lines 16-22: Parse API error, 204, JSON or text.

frontend/src/context/AuthContext.jsx

Purpose: React frontend foundation/component.

Lines 1-16: Global user/loading state and startup /me restore.

Lines 18-26: Save/remove token and user.

Lines 28-35: Provide state and hook.

frontend/src/components/LoadingButton.jsx

Purpose: React frontend foundation/component.

Lines 1-3: Reusable CSS spinner.

Lines 5-9: Disable during loading, expose aria-busy and swap label.

frontend/src/components/ConfirmDialog.jsx

Purpose: React frontend foundation/component.

Lines 1-16: Configurable confirmation props.

Lines 17-20: Choose icon/button tone.

Lines 21-33: Backdrop, dialog semantics, safe close and loading confirm action.

frontend/src/components/ProtectedRoute.jsx

Purpose: React frontend foundation/component.

Lines 1-10: Show loader, redirect signed-out users, block non-admins, otherwise render child.

frontend/src/components/Navbar.jsx

Purpose: React frontend foundation/component.

Lines 1-20: Menu/sign-out state and confirmed logout.

Lines 22-37: Cross-route Pitches navigation and retry scroll.

Lines 39-73: Utility bar, brand, responsive navigation and role-aware links.

Lines 74-76: Custom sign-out confirmation.

frontend/src/components/Footer.jsx

Purpose: React frontend foundation/component.

Lines 1-30: Brand, exploration links, contact details, hours and current-year footer.

frontend/src/components/Reveal.jsx

Purpose: React frontend foundation/component.

Lines 1-20: IntersectionObserver reveals once and cleans up.

Lines 22-24: Apply visible class and optional stagger delay.

frontend/src/components/AiAssistant.jsx

Purpose: React frontend foundation/component.

Lines 1-10: Chat state.

Lines 12-25: Prevent duplicates, append user/bot/error messages and loader.

Lines 27-37: Floating panel, message list, input and toggle.

frontend/src/data.js

Purpose: React frontend foundation/component.

Lines 1-38: Representative fallback pitches for visual/offline development.

Lines 40-47: Contact constants and Bangladeshi currency formatter.

10.11 Frontend pages

frontend/src/pages/Home.jsx

Purpose: Page-level user experience.

Lines 1-18: Imports and amenity content.

Lines 20-36: Load live pitches/reviews, refresh on focus, preserve fallback.

Lines 38-80: Hero, animated sports illustration and CTA.

Lines 82-96: Ticker and quick booking.

Lines 98-119: Dynamic pitch cards and ratings.

Lines 121-170: Experience, amenities, process and final CTA.

frontend/src/pages/AuthPage.jsx

Purpose: Page-level user experience.

Lines 1-19: Registration/login state and navigation context.

Lines 21-26: Request/resend OTP and enter verification stage.

Lines 28-41: Submit OTP or login, save auth and redirect by role.

Lines 43-47: Resend with loader/errors.

Lines 49-75: Two-step form, numeric six-digit input, password visibility and actions.

frontend/src/pages/PasswordPage.jsx

Purpose: Page-level user experience.

Lines 1-13: Read token and hold form/result state.

Lines 15-22: Choose forgot/reset payload and endpoint with loader.

Lines 24-31: Success state or email/password form.

frontend/src/pages/BookingPage.jsx

Purpose: Page-level user experience.

Lines 1-20: Date helpers, twelve fallback slots and time formatting.

Lines 21-37: Pitch/date/slot/coupon/request state.

Lines 39-59: Load pitches, daily slots and seven-day calendar.

Lines 61-69: Validate coupon preview.

Lines 71-94: Require login/details, create booking, skip admin payment, start SSLCommerz or demo flow.

Lines 96-115: Three-step selection UI and live summary.

frontend/src/pages/Dashboard.jsx

Purpose: Page-level user experience.

Lines 1-14: Imports and completion helper including overnight sessions.

Lines 16-45: Bookings/reviews/profile/filter/modal states and data load.

Lines 47-60: Profile save and confirmed cancellation with delay loader.

Lines 63-81: Password verification and final account deletion.

Lines 83-98: Dashboard, search/filter, receipts/reviews/cancel/profile and dialogs.

Lines 100-102: Password modal with visible error.

Lines 104-106: Single printable receipt.

Lines 108-121: Review form with stars and loader.

frontend/src/pages/Admin.jsx

Purpose: Page-level user experience.

Lines 1-44: Imports, helpers and all console/modal/action states.

Lines 46-58: Parallel load and booking filters.

Lines 60-102: Pitch image validation, preview, upload, create, update confirmation and deletion.

Lines 104-135: Manual booking, coupon, messages and booking-decision operations.

Lines 137-161: Tabs, overview, tables, modals and all confirmation dialogs.

Lines 164-180: Wait helper, booking filters/table, coupon table and message list.

Lines 183-213: Pitch, manual booking, coupon and message modal forms.

frontend/src/pages/ContactPage.jsx

Purpose: Page-level user experience.

Lines 1-10: Form/request state.

Lines 12-23: Duplicate guard, API submit, reset and change helper.

Lines 24-31: Contact information and validated form.

frontend/src/pages/PaymentResult.jsx

Purpose: Page-level user experience.

Lines 1-11: Read status and choose success/failure content.

Lines 13-26: Result alert, pending approval badge and navigation.

frontend/src/pages/PaymentSuccess.jsx

Purpose: Page-level user experience.

Lines 1-6: Demo-compatible success and admin-waiting page.

frontend/src/styles.css

Purpose: Complete dark theme, layout, animation, responsive and print design.

Lines 1-33: Fonts, color variables, reset and page shell.

Lines 36-73: Navbar, brand animation and confirmation dialog.

Lines 76-94: Buttons, loaders, links and headings.

Lines 97-138: Hero layers, typography, illustration/orbits/cards and ticker.

Lines 141-181: Quick booking and pitch cards.

Lines 182-260: Experience, amenities, process, CTA and footer.

Lines 263-337: Booking, form, authentication and simple-page layouts.

Lines 340-380: Dashboard and booking cards.

Lines 381-470: Admin console, tables, payment states, cards, messages, modals, uploads and contact page.

Lines 473-524: Calendar, coupons, filters, reviews, receipt, notes and rating styles.

Lines 526-538: A4 print isolation so only one receipt is printed.

Lines 540-549: Core animations.

Lines 551-653: Desktop-to-mobile responsive breakpoints.

Lines 655-659: Reduced-motion accessibility.

Lines 662-730: Final dark visual polish, richer cards, hover depth and corrected process connectors.

Lines 732-740: Small-screen connector adjustments.

Lines 742-763: Football pitch drawing, football and cricket-bat orbits and floating-card positions.

Lines 765-783: Final responsive booking/calendar/receipt corrections.

frontend/vite.config.js

Purpose: Development proxy and dual build destination.

Lines 1-5: Load Vite and React plugin.

Lines 6-9: Run dev server on 5173 and proxy /api to Spring 9091.

Lines 10-13: Build to Vercel dist in production or Spring static resources locally.

frontend/vercel.json

Purpose: SPA refresh support.

Lines 1-3: Rewrite every frontend route to index.html.

frontend/package.json

Purpose: Frontend metadata, scripts and exact direct dependencies.

Lines 1-10: Private ESM project and dev/build/preview scripts.

Lines 11-19: React, icons, Vite and React plugin versions.

frontend/index.html

Purpose: Vite HTML entry.

Lines 1-end: Defines metadata, root element and module entry to src/main.jsx.

10.12 Test files

src/test/java/.../BookingServiceTest.java

Purpose: 9 tests for pricing, duration, collisions, slots/calendar, coupons, admin shortcut and payment approval.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../AuthServiceTest.java

Purpose: 2 tests proving correct OTP creates and incorrect OTP does not.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../MailServiceTest.java

Purpose: 2 tests for OTP/expiry content and deployed reset URL.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../CouponServiceTest.java

Purpose: 3 tests for percentage math, expiry and normalized creation.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../ReviewServiceTest.java

Purpose: 2 tests for completed versus future sessions.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../UserServiceTest.java

Purpose: 2 tests for incorrect/correct account-delete password.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../SecurityConfigTest.java

Purpose: 2 tests separating gateway callback and ordinary API CORS.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../JwtServiceTest.java

Purpose: 1 test for token generation and identity validation.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../ContactServiceTest.java

Purpose: 1 test for workflow status and trimmed note.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

src/test/java/.../AiAssistantServiceTest.java

Purpose: 1 test for fallback without provider credentials.

Lines Imports and setup: JUnit 5 defines tests; Mockito mocks collaborators; InjectMocks builds the service.

Lines Arrange: Create entities/requests and program mock responses.

Lines Act: Call one production method.

Lines Assert: Check return/state/error and verify repository/service interactions.

10.13 Generated and binary files

File/group	How to explain it
frontend/package-lock.json	npm-generated dependency tree that makes installations reproducible; do not hand-edit.
src/main/resources/static/assets/*	Vite-generated minified JavaScript/CSS. Source of truth is frontend/src; rebuild instead of editing.
src/main/resources/static/index.html	Generated production HTML copied by Vite for Spring Boot serving.
frontend/public/images/*.png	AI-generated/static visual assets, copied to both deployment outputs; binary files have no source lines.
target/* and frontend/node_modules/*	Local build outputs/dependencies and intentionally not project source.

11. Common viva questions and short answers

Q: Why use DTOs instead of returning models?

A: DTOs expose only intended fields, keep validation at the API boundary, and prevent password/reset data from leaking.

Q: Why is the backend stateless?

A: JWT travels with each request, so the server does not need an HTTP session and works with a separate Vercel frontend.

Q: How is the password protected?

A: BCrypt creates a salted slow hash. Login uses PasswordEncoder.matches; plain passwords are never stored.

Q: Is the OTP stored in plain text?

A: No. SecureRandom creates it, the email receives it, and MongoDB stores SHA-256(email:OTP) with expiry and attempt count.

Q: Why return the same forgot-password message?

A: It prevents attackers from discovering which email addresses have accounts.

Q: How does a production reset link avoid localhost?

A: MailService prefixes it with app.frontend-url, which Render receives from FRONTEND_URL.

Q: What stops two users booking one slot?

A: The service rechecks a non-cancelled booking and synchronizes creation in the current single-instance design.

Q: Why are payment and booking statuses separate?

A: Payment can be verified while the venue still needs to approve the reservation.

Q: Why not trust success_url?

A: Browser callbacks can be forged. The server verifies val_id, status, transaction ID and amount with SSLCommerz.

Q: What makes callbacks idempotent?

A: Already-PAID bookings return success without repeating transitions, and couponCounted blocks duplicate coupon usage.

Q: How do admin bookings differ?

A: They become CONFIRMED immediately and the frontend skips payment; manual bookings can also be marked paid.

Q: How do pitch edits reach the home page?

A: PitchService evicts caches and Home fetches live API data, including a refresh on browser focus.

Q: Why use FormData for images?

A: Binary multipart upload is appropriate; api.js intentionally does not force JSON content type for FormData.

Q: What happens without OpenAI?

A: The assistant returns deterministic answers, so the application still works.

Q: How are duplicate submissions prevented?

A: Handlers check a busy flag, LoadingButton is disabled, and destructive actions require a separate confirmation state.

Q: What does @Cacheable do?

A: It stores a method result under a key; later equal calls return memory data until five-minute expiry or eviction.

Q: Why is CSRF disabled?

A: The API uses bearer tokens rather than cookie authentication. CORS and JWT still protect browser/API access.

Q: What is CORS?

A: A browser rule controlling which origins may call the API. Normal routes allow the Vercel origin; payment callbacks have a separate rule.

Q: What does @Valid do?

A: It triggers constraints on the DTO before controller logic, and GlobalExceptionHandler returns readable field errors.

Q: What is dependency injection?

A: Spring creates services/repositories and Lombok-generated constructors receive them, making code modular and testable.

12. Demonstration checklist and honest limitations

Suggested five-minute teacher demonstration

- Open the landing page and explain live pitches, animations and responsive navbar.
- Open registration, request an OTP, and point out that no real user exists before verification.
- Sign in, choose a pitch/date/slot, apply a coupon and explain the 1.5-hour price.
- Start SSLCommerz sandbox checkout and explain server-side validation and AWAITING_APPROVAL.
- Open the admin panel, filter paid bookings and confirm one with the custom dialog/loader.
- Edit a pitch and show that cache eviction plus live fetch updates public cards.
- Open a user receipt and show the single-page print view.
- Show tests passing and identify the deployed Vercel and Render responsibilities.

Honest classroom-project limitations

- synchronized protects one Java process; multiple Render instances would need a unique database constraint or distributed lock.
- Uploaded images use local disk. Render's ephemeral filesystem can lose files after redeploy; production should use Cloudinary, S3 or persistent disk.
- Pending registration documents are checked for logical expiry but are not currently removed by a MongoDB TTL index.
- JWT is stored in localStorage for simplicity. A hardened consumer system may prefer short-lived tokens and secure HttpOnly refresh cookies.
- Rejecting a paid booking records cancellation but does not automatically issue an SSLCommerz refund.
- The custom router is intentionally small and does not implement every feature of React Router.

Final summary: Turfly demonstrates a complete, explainable full-stack workflow: validated React forms, REST controllers, service-layer rules, MongoDB persistence, JWT security, email verification, real hosted payment validation, admin approval, caching, optional AI, tests, and split cloud deployment.